Tutor

Mauro Farina

- Dottorando in ingegneria industriale e dell’informazione
- 2° anno
- Misure di Internet
- mauro.farina@phd.units.it

Repository esercitazioni

- github.com/mauro-farina/database-2026

Esercitazioni

4 / 5 esercitazioni tra marzo e maggio:

- Martedì 10 marzo
- TBD
- TBD
- TBD
- [TBD]

Design database

Design database

1. Formulazione del problema
    a. Requisiti
2. Sviluppo del modello logico
    a. Tabelle
    b. Vincoli
3. Implementazione

Problema

Tenere traccia degli esami sostenuti dagli studenti per il corrente anno accademico

- Libretto esami superati da uno studente
- Controllare quale professore insegna ciascun corso

Tabelle?

Tenere traccia degli esami sostenuti dagli studenti per il corrente anno accademico

- Libretto esami superati da uno studente
- Controllare quale professore insegna ciascun corso

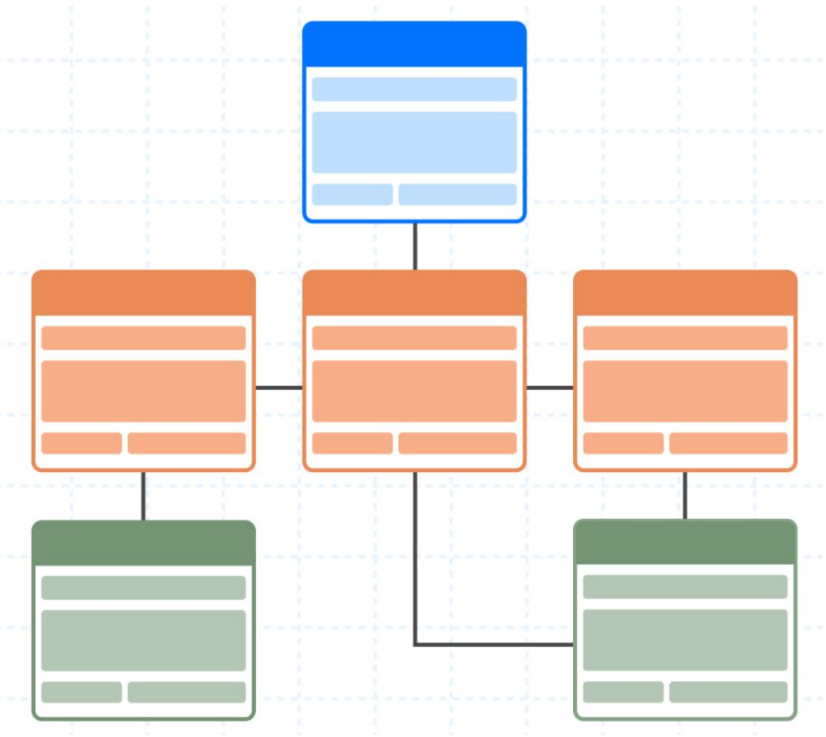


Tabelle (1)

Studenti

Esami

Corsi

Professori

Tabelle (2)

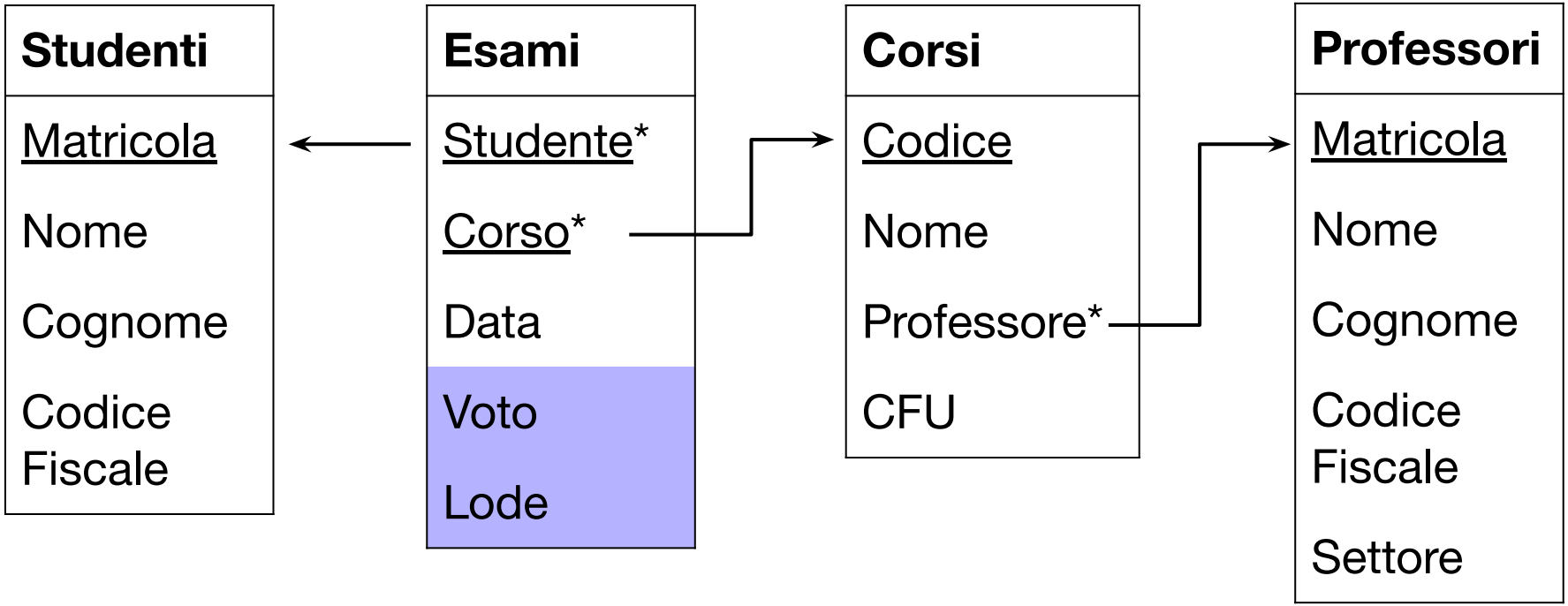
Studenti
Matricola
Nome
Cognome
Codice Fiscale

Esami
Studente
Corso
Data
Voto
Lode

Corsi
Codice
Nome
Professore
CFU

Professori
Matricola
Nome
Cognome
Codice Fiscale
Settore

Tabelle (3)


Studenti
Matricola
Nome
Cognome
Codice Fiscale
Esami
Studente*
Corso*
Data
Voto
Lode
Corsi
Codice
Nome
Professore*
CFU
Professori
Matricola
Nome
Cognome
Codice Fiscale
Settore

Implementazione database

Implementazione database

1. Creazione del database

```
CREATE DATABASE unidb;
```

2. Selezione del database

```
USE unidb;
```

Tabella `studenti`

```
CREATE TABLE studenti(
 matricola CHAR(9)
 nome      VARCHAR(50)
 cognome   VARCHAR(50)
 cf        CHAR(16)
);
```

Studenti
<u>Matricola</u>
Nome
Cognome
Codice Fiscale

Tabella `studenti`

```
CREATE TABLE studenti(
 matricola CHAR(9)       PRIMARY KEY,
 nome      VARCHAR(50)   NOT NULL,
 cognome   VARCHAR(50)   NOT NULL,
 cf        CHAR(16)      UNIQUE NOT NULL
);
```

Studenti
<u>Matricola</u>
Nome
Cognome
Codice Fiscale

Tabella `professori`

```
CREATE TABLE professori(
 matricola INT
 nome      VARCHAR(50)
 cognome   VARCHAR(50)
 cf        CHAR(16)
 settore   VARCHAR(12)
 PRIMARY KEY (matricola)
);
```

Professori
<u>Matricola</u>
Nome
Cognome
Codice Fiscale
Settore

Tabella professori

```
CREATE TABLE professori(
 matricola INT          AUTO_INCREMENT,
 nome      VARCHAR(50)  NOT NULL,
 cognome   VARCHAR(50)  NOT NULL,
 cf        CHAR(16)     UNIQUE NOT NULL,
 settore   VARCHAR(12)  NOT NULL,
 PRIMARY KEY (matricola)
);
```

Professori
<u>Matricola</u>
Nome
Cognome
Codice Fiscale
Settore

Tabella `corsi`

```
CREATE TABLE corsi(
 codice      CHAR(5)       PRIMARY KEY,
 nome        VARCHAR(100)  NOT NULL,
 professore  INT           NOT NULL,
 cfu         TINYINT       NOT NULL
);
```

Corsi
<u>Codice</u>
Nome
Professore*
CFU

Tabella `corsi`

```
CREATE TABLE corsi(
 codice      CHAR(5)       PRIMARY KEY,
 nome        VARCHAR(100) NOT NULL,
 professore INT           NOT NULL,
 cfu         TINYINT       NOT NULL,
 FOREIGN KEY (professore)
  REFERENCES professori(codice)

);
```

Corsi
<u>Codice</u>
Nome
Professore*
CFU

Tabella `corsi`

```
CREATE TABLE corsi(
 codice      CHAR(5)       PRIMARY KEY,
 nome        VARCHAR(100) NOT NULL,
 professore INT           NOT NULL,
 cfu         TINYINT       NOT NULL,
 FOREIGN KEY (professore)
  REFERENCES professori(codice)
  ON DELETE SET NULL
);
```

Corsi
<u>Codice</u>
Nome
Professore*
CFU

Tabella `esami`

```
CREATE TABLE esami(
 corso CHAR(5), studente CHAR(9),
 data DATE NOT NULL, voto TINYINT NOT NULL,
 lode BOOL DEFAULT FALSE,
 PRIMARY KEY (corso, studente),
 FOREIGN KEY (corso) REFERENCES corsi(codice)

 FOREIGN KEY (studente) REFERENCES

);
```

Esami
Studente*
Corso*
Data
Voto
Lode

Tabella esami

```
CREATE TABLE esami(
 corso CHAR(5), studente CHAR(9),
 data DATE NOT NULL, voto TINYINT NOT NULL,
 lode BOOL DEFAULT FALSE,
 PRIMARY KEY (corso, studente),
 FOREIGN KEY (corso) REFERENCES corsi(codice)

 FOREIGN KEY (studente) REFERENCES
  studenti(matricola) ON DELETE CASCADE
);
```

Esami
Studente*
Corso*
Data
Voto
Lode

Tabella esami

```
CREATE TABLE esami(
 corso CHAR(5), studente CHAR(9),
 data DATE NOT NULL, voto TINYINT NOT NULL,
 lode BOOL DEFAULT FALSE,
 PRIMARY KEY (corso, studente),
 FOREIGN KEY (corso) REFERENCES corsi(codice)
 FOREIGN KEY (studente) REFERENCES
  studenti(matricola) ON DELETE CASCADE,
 CHECK (voto BETWEEN 18 AND 30),
 CHECK (lode=FALSE OR (lode=TRUE AND voto=30))
);
```

Esami
Studente*
Corso*
Data
Voto
Lode

Comandi utili (1)

Tabelle presenti nel database: `SHOW TABLES;`

Tables_in_unidb
corsi
esami
professori
studenti

Comandi utili (2)

Descrizione delle colonne di una tabella: `DESCRIBE esami;`

#	Field	Type	Null	Key	Default
1	corso	char(5)	NO	PRI	NULL
2	studente	char(9)	NO	PRI	NULL
3	data	date	NO		NULL
4	voto	tinyint	NO		NULL
5	lode	tinyint(1)	YES		0

Comandi utili (3)

Codice usato da per creare le tabelle: `SHOW CREATE TABLE esami;`

```
...
CONSTRAINT `esami_ibfk_2`
  FOREIGN KEY (`studente`)
  REFERENCES `studenti` (`matricola`)
  ON DELETE CASCADE,
CONSTRAINT `esami_chk_1`
  CHECK ((`voto` between 18 and 30)),
...
```

Creiamo e popoliamo `unidb`

Repository GitHub → [`github.com/mauro-farina/database-2026`](https://github.com/mauro-farina/database-2026)

- `unidb.sql`
  - 301 studenti
  - 52 professori
  - 31 corsi
  - 2203 esami

Per eseguire lo script:

- Copia-incolla, oppure
- Da MySQL Workbench: File > Open SQLScript

Query

Soluzioni

Per i successivi esercizi, caricare la soluzione (**codice SQL**) sull'apposito Google Form:

- `github.com/mauro-farina/database-2026`
- `forms.gle/X7Zn2fXr1VgLmKyP9`

Esercizio 1

Trovare il numero di studenti iscritti alla triennale di ingegneria informatica che sono nati a gennaio, febbraio, o marzo.

- Matricole di ingegneria informatica iniziano con IN05
- Nel codice fiscale il mese di nascita è mappato al 9° carattere:
  - `RSSMRC03A21L424F` → Gennaio
  - `BNCFNC03B47L483B` → Febbraio
  - `TRVNDR03C11D969H` → Marzo

Esercizio 1: Soluzione (1)

Trovare il numero di studenti iscritti alla triennale di ingegneria informatica che sono nati a gennaio, febbraio, o marzo.

```
SELECT count(*) FROM studenti
WHERE Matricola LIKE "IN05%"
   AND (cf LIKE "________A%"
        OR cf LIKE "________B%"
        OR cf LIKE "________C%");
```

Funzione substring()

```
substring(cf, N, M)
```

Restituisce la sottostringa di `cf` di lunghezza `M` che alla posizione `N`

```
SELECT substring("RSSMRC03A21L424F", 1, 3);
-- "RSS"

SELECT substring("RSSMRC03A21L424F", 7, 2);
-- "03"
```

Esercizio 1: Soluzione (2)

Trovare il numero di studenti iscritti alla triennale di ingegneria informatica che sono nati a gennaio, febbraio, o marzo.

```
SELECT count(*) FROM studenti
WHERE Matricola LIKE "IN05%"
   AND substring(cf, 9, 1) IN ("A", "B", "C");
```

Esercizio 1: Soluzione (3)

Trovare il numero di studenti iscritti alla triennale di ingegneria informatica che sono nati a gennaio, febbraio, o marzo.

```
SELECT count(*) FROM studenti
WHERE Matricola LIKE "IN05%"
   AND substring(cf, 9, 1) BETWEEN "A" AND "C";
```

Esercizio 2

Elencare nome e cognome degli studenti che hanno preso 30 e lode nell'esame "Basi di Dati"

Risultato attesa:

- Enea Ghini
- Sofia Cagnotti

Esercizio 2: Soluzione

Elencare nome e cognome degli studenti che hanno preso 30 e lode nell'esame "Basi di Dati"

```
SELECT s.nome, s.cognome
FROM studenti s
INNER JOIN esami e ON s.matricola = e.studente
INNER JOIN corsi c ON e.corso = c.codice
WHERE c.nome="Basi di Dati" AND e.lode IS TRUE;
```

Esercizio 3

Elencare i professori che **non** insegnano nessun corso

Risultato atteso (28):

- Valentina Brinzi
- Marika Linzi
- Tancredi Santana moscato
- …
- Amelia Taurore

Esercizio 3: Soluzione (1)

Elencare i professori che **non** insegnano nessun corso

```
SELECT * FROM professori p
LEFT OUTER JOIN corsi c
ON p.matricola=c.professore
WHERE c.nome IS NULL;
```

Esercizio 3: Soluzione (2)

Elencare i professori che **non** insegnano nessun corso

```
SELECT * FROM professori
WHERE matricola NOT IN (
    SELECT DISTINCT professore
    FROM corsi
    WHERE professore IS NOT NULL
);
```

https://www.geeksforgeeks.org/sql/how-to-use-null-values-inside-not-in-clause-in-sql/

Esercizio 4

Contare il numero di studenti per ciascun corso di laurea (in ordine decrescente)

- Il corso di laurea è dato dai primi 4 caratteri della matricola
- `substring(colonna, indice, lunghezza)`

Risultato atteso (8):

- EC21 46
- IN05 45
- …
- EC72 29

Esercizio 4: Soluzione

Contare il numero di studenti per ciascun corso di laurea (in ordine decrescente)

```
SELECT
   substring(matricola, 1, 4) AS cdl,
   count(*) AS num
FROM studenti
GROUP BY cdl
ORDER BY num DESC;
```